WEEK 4 • DAY 2

ML Workflow & Data Preprocessing

Learn • Build • Debug • Explain

Date	27-Oct-2026	**Duration**	5 hours
Difficulty	Intermediate	**Deliverable**	Preprocessing notebook

1. Learning Goals

- Describe the ML workflow from problem definition to evaluation.
- Find missing values, duplicates, inconsistent categories, and unsuitable columns.
- Explain why and when numerical features need scaling.
- Encode categorical columns into model-friendly numeric features.
- Prevent data leakage by splitting data before fitting preprocessing steps.
- Build and test a reusable scikit-learn preprocessing pipeline.

2. Concepts in Simple Words

ML workflow: Define the business problem → collect data → inspect and clean → split data → fit preprocessing on training data → train a model → evaluate on held-out data → improve carefully.

Preprocessing: Preparing raw data for a model, such as handling missing values, encoding categories, scaling numeric columns, and checking invalid records.

Missing values: Blank or unavailable entries. Numeric columns can often use the training-set median; categorical columns can use the most frequent value or an explicit Unknown category.

Scaling: Changing numeric features to comparable scales. StandardScaler standardises using training-set statistics; MinMaxScaler commonly maps values to 0–1. Scaling is useful for many distance-based and gradient-based methods, but not every model needs it.

Encoding: Representing categories numerically. One-hot encoding creates indicator columns. Avoid arbitrary numeric codes for unordered categories such as city names.

Data leakage: When information unavailable at prediction time, or information from test data, influences training and makes evaluation misleadingly optimistic.

Pipeline: A sequence of preprocessing and model steps fitted together so the same transformations are applied consistently.

fit vs transform: fit learns values/rules (such as a median or scaling statistics); transform applies them. Fit preprocessing on training data only, then transform both train and test data.

3. Suggested Five-Hour Plan

Time	Activity
Hour 1	Understand workflow and inspect the sample dataset.
Hour 2	Find missing values, duplicates, inconsistent categories, and target column.
Hour 3	Practise imputation, one-hot encoding, and scaling.
Hour 4	Build a ColumnTransformer and Pipeline; split data correctly.
Hour 5	Debug mistakes, explain leakage, and finish the notebook.

4. Sample Dataset

Create customer_churn_small.csv from the rows below, or build the same data in a pandas DataFrame. This tiny dataset is for workflow practice only; it is not suitable for meaningful model-performance conclusions.

age	monthly_spend	city	plan_type	support_tickets	churn
24	500	Hyderabad	Basic	2	Yes
35	1200	Chennai	Premium	0	No
29	(blank)	Hyderabad	Standard	1	No
42	2200	Bengaluru	Premium	3	Yes
31	900	Chennai	Basic	(blank)	No
(blank)	(blank)	Hyderabad	Basic	4	Yes
27	750	Visakhapatnam	Standard	1	No
39	1800	Bengaluru	Premium	2	No
33	1100	Chennai	Standard	0	No
46	2500	Hyderabad	Premium	5	Yes
28	650	Bengaluru	Basic	1	No
36	1500	Chennai	Standard	2	Yes

Target: churn. **Candidate features:** age, monthly_spend, city, plan_type, support_tickets. Never include churn in X when predicting churn.

5. Build the Preprocessing Notebook

Create one notebook named Week4_Day2_Preprocessing_YourName.ipynb. Complete these steps in order.

Step 1 — Load and inspect: Load the data. Show head(), shape, info(), missing-value counts, duplicate count, and churn distribution.

Step 2 — Define X and y: Set y = df['churn']. Set X to the five candidate feature columns only. Explain why churn must not be in X.

Step 3 — Split first: Use train_test_split(test_size=0.25, random_state=42). Consider stratify=y if class counts allow it; mention that the tiny dataset limits reliable evaluation.

Step 4 — Identify column types: Numeric: age, monthly_spend, support_tickets. Categorical: city, plan_type. Confirm data types and missing values.

Step 5 — Numeric preprocessing: Use SimpleImputer(strategy='median') followed by StandardScaler().

Step 6 — Categorical preprocessing: Use SimpleImputer(strategy='most_frequent') followed by OneHotEncoder(handle_unknown='ignore').

Step 7 — Combine transformations: Use ColumnTransformer to apply the numeric and categorical pipelines to their correct columns.

Step 8 — Fit on training data only: Fit the preprocessor on X_train, then transform X_train and X_test. Never fit preprocessing on the full dataset before splitting.

Step 9 — Optional full ML pipeline: Add LogisticRegression(max_iter=1000) after the preprocessor. Fit on training data and evaluate on the test set. Treat scores as a demonstration only.

Step 10 — Explain: State which columns were imputed, scaled, and encoded, and why test data did not teach the preprocessing steps.

6. Code Starter (Complete the TODOs)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

df = pd.read_csv("customer_churn_small.csv")
numeric_features = ["age", "monthly_spend", "support_tickets"]
categorical_features = ["city", "plan_type"]

X = df[numeric_features + categorical_features]
y = df["churn"]

# TODO 1: Split into X_train, X_test, y_train, y_test.
# Use test_size=0.25 and random_state=42.

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.25, random_state=42, stratify=y

)

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])

# TODO 2: Fit preprocessor on X_train only, then transform
# both X_train and X_test.
```

```
X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

# TODO 3 (optional): Build Pipeline([
#   ("preprocessor", preprocessor),
#   ("model", LogisticRegression(max_iter=1000))
# ]) and fit it using training data.

model_pipeline = Pipeline(steps=[

    ("preprocessor", preprocessor),

    ("model", LogisticRegression(max_iter=1000))

])

model_pipeline.fit(X_train, y_train)

# TODO 4: Evaluate on X_test and y_test. Explain limitations.

y_pred = model_pipeline.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:")

print(classification_report(y_test, y_pred, zero_division=0))

# Limitation: The dataset has only 12 rows,

# so the evaluation score is not reliable.
```

7. Debugging: Identify and Fix the Mistakes

A. Scaling before splitting — A student scales all rows, then splits into train and test.

Your task: Why can this leak information? What order should be used?

Scaling before splitting can cause data leakage because test data influences the scaling values. Split first, then fit the scaler on training data only.

B. Target inside X — A student includes churn in X and also uses churn as y.

Your task: Explain target leakage and correct the feature list.

Including churn in X causes target leakage because the model already receives the answer it must predict. Keep churn only in y.

C. Unknown category — A test row has a city not seen during training and the encoder raises an error.

Your task: Which OneHotEncoder option helps handle this?

Use OneHotEncoder(handle_unknown="ignore") to handle categories not seen during training.

D. Wrong imputation — The student fills missing monthly_spend with the text 'unknown' and then scales the column.

Your task: Why does this cause a type problem? Suggest a numeric strategy.

The text "unknown" cannot be scaled as a number. Use SimpleImputer(strategy="median") for numeric columns.

E. Fitting on test data — The student calls preprocessor.fit_transform(X_test) after training preprocessing.

Your task: Why should the already-fitted preprocessor transform test data?

Use preprocessor.transform(X_test) because the preprocessor has already learned its values from training data.

8. Explain in Your Own Words

1. What is the difference between fit, transform, and fit_transform?
2. Why split data before fitting imputation or scaling?
3. When is scaling useful, and why might some models not need it?
4. What does one-hot encoding do? Why can numeric codes for city names mislead a model?
5. Give one realistic example of data leakage.
6. Why is a Pipeline useful?

1.Answer: fit(): Learns patterns or statistics from the data.

transform(): Uses the learned information to change the data.

fit_transform(): Learns and transforms the data in one step.

2.Answer:We split data before fitting imputation or scaling to prevent data leakage and ensure that the test data does not influence the training process.

3.Answer:Scaling is useful when features have different ranges, especially for KNN, SVM, and Logistic Regression. Decision Trees and Random Forests usually do not need scaling because they make decisions based on feature values rather than distances.

4.Answer: One-hot encoding creates separate 0/1 columns for each category. Numeric city codes can mislead a model into thinking cities have a numerical order.

5.Answer:Using a student's final exam marks to predict whether they passed is data leakage because the final marks reveal the answer.

6.Answer:A Pipeline combines data preprocessing and model training into one workflow, making the code easier to manage and helping prevent data leakage.

9. Daily Deliverable & Checklist

Submit ONE notebook: Week4_Day2_Preprocessing_YourName.ipynb

- Notebook runs from top to bottom without unexplained errors.
- Includes data inspection and missing-value / duplicate checks.

- Defines X and y separately.
- Splits data before fitting preprocessing.
- Uses imputation, scaling, one-hot encoding, and ColumnTransformer.
- Fits transformations only on training data and applies them to test data.
- Includes debugging answers and own-words explanations.
- Ends with a brief learning summary and limitations.

10. Assessment Rubric (100 Marks)

Assessment area	**Marks**
Dataset inspection and feature/target definition	15
Correct split and leakage prevention	20
Imputation, scaling, and encoding	25
ColumnTransformer / Pipeline implementation	20
Debugging, explanations, notebook clarity	20
TOTAL	**100**

Trainer note: The sample is deliberately small and contains missing values. Its model score is not statistically meaningful; assess the correctness of the workflow and leakage prevention.